

Topic 1 - Line Chart

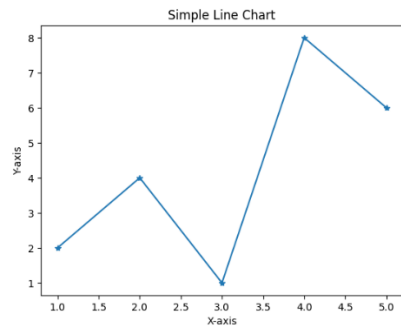
- A line chart is a simple graph used to show changes in data over time.
- It displays data as points that are connected using straight lines.
- The horizontal axis (x-axis) usually shows time, such as days, months, or years.
- The vertical axis (y-axis) represents the values being measured, like marks, temperature, or sales.
- Line charts make it easy to see trends, such as increase, decrease, or no change.
- They are very useful for beginners because they are easy to read and understand.
- For example, a line chart can show how internet usage increases each year or how temperature changes during a day.

Program:

a. Using Matplotlib:

```
import matplotlib.pyplot as plt
# Custom data
x = [1, 2, 3, 4, 5]
y = [2, 4, 1, 8, 6]
# Create line chart
plt.plot(x, y, marker='*')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.title('Simple Line Chart')
# Display the chart
plt.show()
```

Output:



b. Using Seaborn:

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

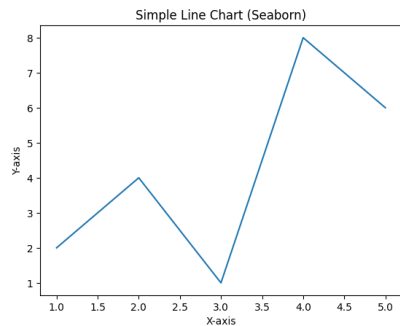
# Custom data
d = pd.DataFrame({
    'x': [1, 2, 3, 4, 5],
    'y': [2, 4, 1, 8, 6]
})

# Create line chart using seaborn
sns.lineplot(data=d, x='x', y='y')

# Add labels and title (using matplotlib functions)
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.title('Simple Line Chart (Seaborn)')

# Display the chart
plt.show()
```

Output:



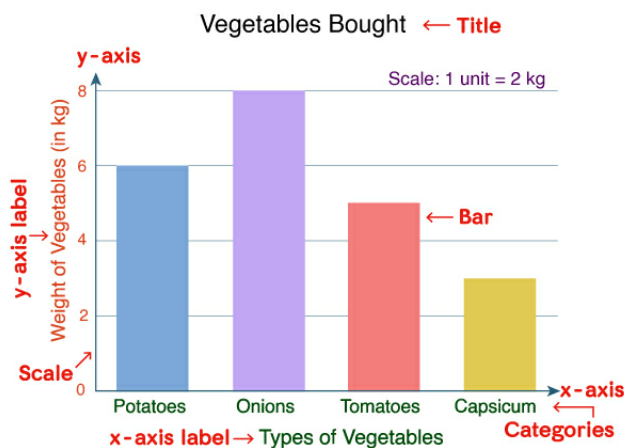
Topic 2 - Bar Chart

- A bar chart is a graph that uses rectangular bars to show data.
- Each bar represents a category (like subjects, products, or years).
- The height or length of a bar shows the value or amount.
- Taller/longer bar = bigger value.
- Shorter bar = smaller value.

Parts of a Bar Graph

MATH MONKS

A bar graph, also called the bar chart, is a way of representing data graphically in the form of bars.



Parts of a Bar Chart

- Title – tells what the chart is about.
- X-axis (horizontal) – shows categories (e.g., names, types).
- Y-axis (vertical) – shows numbers or values.
- Bars – show the data for each category.
- Scale – helps read exact values (e.g., 0, 10, 20).

Types of Bar Charts

- Vertical bar chart – bars go up and down.

- Horizontal bar chart – bars go left to right.

Program:

- a. Using Matplotlib:

```
import matplotlib.pyplot as plt
```

```
# Custom data
```

```
categories = ['Apples', 'Bananas', 'Cherries', 'Dates']
```

```
values = [23, 45, 56, 78]
```

```
# Create bar chart
```

```
plt.bar(categories, values)
```

```
# Add labels and title
```

```
plt.xlabel('Fruit')
```

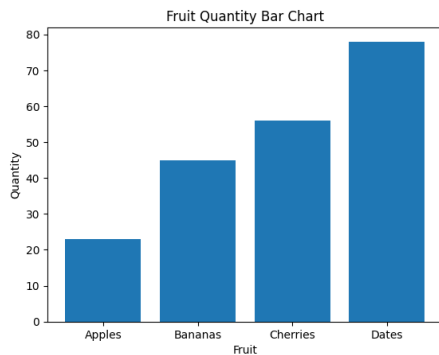
```
plt.ylabel('Quantity')
```

```
plt.title('Fruit Quantity Bar Chart')
```

```
# Show the plot
```

```
plt.show()
```

Output:



- Note: plot() method can also be used and “kind” parameter should be used as “kind=bar”
- Note: Use barh() method to get horizontal bars

Note:

- For getting horizontal bars using matplotlib use barh().
- For getting horizontal bars using seaborn use orient=h attribute and also swap the x and y columns names as per your choice.

b. Using Seaborn:

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

```
import pandas as pd
```

```
# Custom data as a DataFrame (Seaborn works best with DataFrames)
```

```
d = pd.DataFrame({
    'Fruit': ['Apples', 'Bananas', 'Cherries', 'Dates'],
    'Quantity': [23, 45, 56, 78]
})
```

```
# Create bar chart using seaborn
```

```
sns.barplot(x='Fruit', y='Quantity', data=d)
```

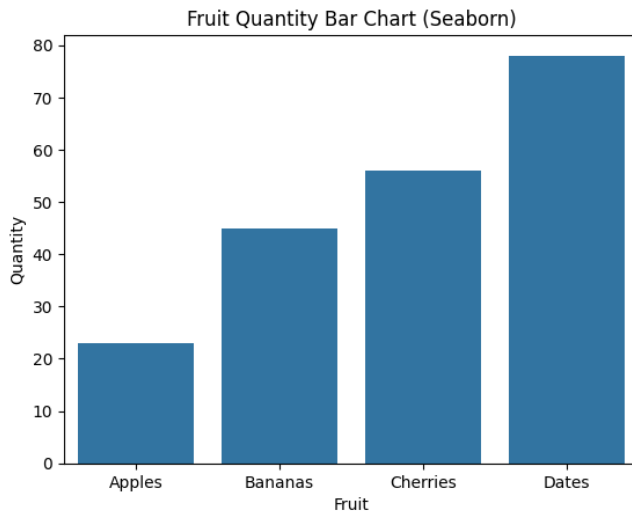
```
# Add labels and title (using matplotlib functions)
```

```
plt.xlabel('Fruit')
```

```
plt.ylabel('Quantity')
plt.title('Fruit Quantity Bar Chart (Seaborn)')
```

```
# Show the plot
plt.show()
```

Output:



Topic 3 - Scatter plot

A scatter plot is a simple graph used to see how two number values are related. It shows data using dots on a graph. Each dot stands for one data item and is placed based on its values on two lines called axes.

- A scatter plot uses dots to show data.
- The X-axis (horizontal line) shows one value.
- The Y-axis (vertical line) shows another value.
- Each dot shows one pair of values.
- If dots go upwards, both values increase together.
- If dots go downwards, one value increases while the other decreases.
- If dots are random, there is no clear relation.
- A dot far from others is called an outlier.
- Scatter plots help us find patterns easily.

In short, a scatter plot helps us understand relationships between two things by just

looking at how the dots are placed on the graph.

Program:

a. Using Matplotlib:

```
import matplotlib.pyplot as plt
```

```
# Custom data
```

```
x = [1, 2, 3, 4, 5, 6, 7, 8]
```

```
y = [2, 5, 3, 8, 7, 10, 9, 12]
```

```
# Create scatter plot
```

```
plt.scatter(x, y, color='blue', label='Data Points')
```

```
# Add labels and title
```

```
plt.xlabel('X Values')
```

```
plt.ylabel('Y Values')
```

```
plt.title('Scatter Plot using Matplotlib')
```

```
# Optional: Add grid and legend
```

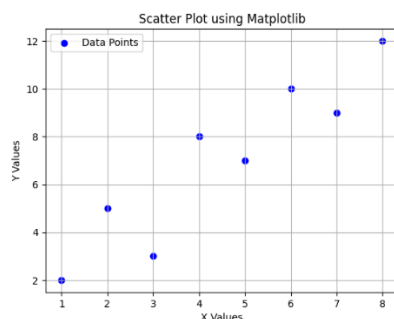
```
plt.grid(True)
```

```
plt.legend()
```

```
# Show plot
```

```
plt.show()
```

Output:



- Note: plot() method can also be used and “kind” parameter should be used as “kind=scatter”

b. Using Seaborn:

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

```
import pandas as pd
```

```
# Custom data as a DataFrame (Seaborn works best with DataFrames)
```

```
d = pd.DataFrame({  
    'X': [1, 2, 3, 4, 5, 6, 7, 8],  
    'Y': [2, 5, 3, 8, 7, 10, 9, 12]  
})
```

```
# Create scatter plot
```

```
sns.scatterplot(data=d, x='X', y='Y', color='green')
```

```
# Add labels and title
```

```
plt.xlabel('X Values')
```

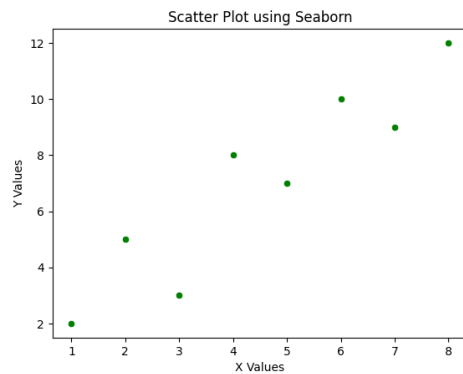
```
plt.ylabel('Y Values')
```

```
plt.title('Scatter Plot using Seaborn')
```

```
# Show plot
```

```
plt.show()
```

Output:



Topic 4 & 5 - Area plot & Stacked Area Plot

An area plot is a graph that shows how data changes over time. It is similar to a line graph, but the space below the line is filled with color. This makes it easy to see whether the data is increasing or decreasing and how big the values are.

- An area plot shows change over time
- It has a filled color area under the line
- X-axis shows time or categories
- Y-axis shows numbers or values
- The colored area helps to see growth clearly

Program:

a. Using Matplotlib:

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
# — Custom Data —
```

```
months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug']
```

```
series_a = [5, 6, 7, 8, 9, 11, 13, 12]
```

```
series_b = [2, 3, 4, 5, 6, 6, 7, 8]
```

```
series_c = [1, 1, 2, 2, 3, 4, 4, 5]
```

```
x = np.arange(len(months))
```

```
# =====
```

```

# AREA PLOT (Single Series)
# =====

plt.figure(figsize=(8,4))
plt.fill_between(x, series_a, color='orange', alpha=0.3)

plt.xticks(x, months)
plt.title("Area Plot - Series A")
plt.xlabel("Month")
plt.ylabel("Value")
plt.grid(alpha=0.3)
plt.show()

# =====

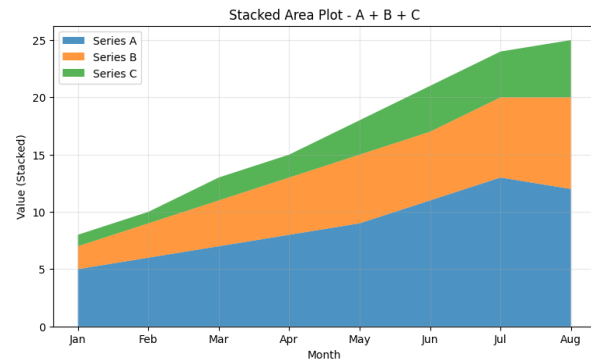
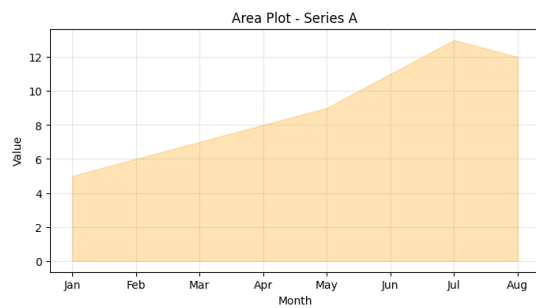
# STACKED AREA PLOT (Multiple Series)
# =====

plt.figure(figsize=(9,5))
plt.stackplot(x, series_a, series_b, series_c,
              labels=['Series A', 'Series B', 'Series C'],
              alpha=0.8)

plt.xticks(x, months)
plt.title("Stacked Area Plot - A + B + C")
plt.xlabel("Month")
plt.ylabel("Value (Stacked)")
plt.legend(loc="upper left")
plt.grid(alpha=0.3)
plt.show()

```

Output:



Alpha Values:

alpha	Effect
1.0	Fully solid grid lines (dark)
0.5	Semi-transparent
0.3	Light, soft grid lines
0.0	Completely invisible

Note: Using seaborn library directly area plot and stacked area plot cannot be visualized

Topic 6 - Pie Chart

- A pie chart is a circular chart that is divided into slices.
- Each slice shows how much a part contributes to the whole.
- The entire pie represents 100%, and every slice is a percentage of that total.
- Pie charts are mainly used when you want to compare parts of a single dataset and understand their proportions easily at a glance.
- A pie chart is round in shape

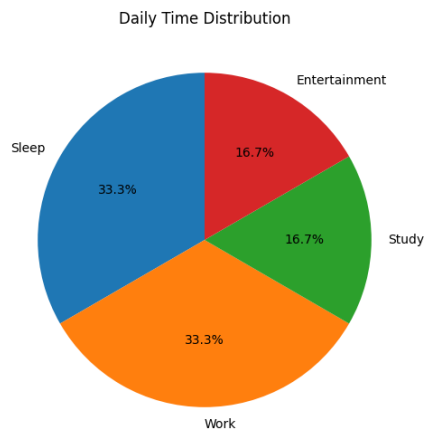
- It is divided into slices
- Each slice represents a category
- The size of a slice depends on its value
- Bigger value bigger slice
- All slices together make 100%
- Best used when:
 - o Number of categories is small
 - o You want to show percentage distribution

Program:

a. Using Matplotlib:

```
import matplotlib.pyplot as plt
# Data
labels = ['Sleep', 'Work', 'Study', 'Entertainment']
sizes = [8, 8, 4, 4]
# Create pie chart
plt.figure(figsize=(6,6))
plt.pie(sizes, labels=labels, autopct='%1.1f%%', startangle=90)
# Title
plt.title('Daily Time Distribution')
# Show chart
plt.show()
```

Output:



Note: Seaborn does not have a direct pie chart function.

Topic 7 - Table Chart

A table chart is a way of showing data in the form of rows and columns, just like a table you see in books or Excel.

It helps us organize information clearly so that anyone can easily read and understand the data.

Instead of showing data using bars or lines, a table chart directly displays values in cells.

We use table charts when:

- Exact values are important
- We want to compare data clearly
- The audience is beginners or non-technical users

Key Features of a Table Chart

- Data is shown in rows and columns
- Each row represents a record
- Each column represents a feature or category
- Easy to read and understand
- No calculations are required to interpret values

Simple Example of a Table Chart

Imagine a table showing student marks:

Student	Math	Science	English
A	85	78	90
B	72	80	75
C	88	85	92

This is a table chart because:

- Rows Students
- Columns Subjects
- Values Marks

Program:

a. Using Matplotlib:

```
import matplotlib.pyplot as plt
```

```
import pandas as pd
```

```
data = {
```

```
    'Math': [85, 72, 88],
```

```
    'Science': [78, 80, 85],
```

```
    'English': [90, 75, 92]
```

```
}
```

```
df = pd.DataFrame(data)
```

```
fig, ax=plt.subplots()
```

```
ax.axis('off')
```

```
t = ax.table(
```

```
    cellText=df.values,
```

```
    colLabels=df.columns,
```

```
    loc='center')
```

```
t.scale(1,2)
```

```
plt.show()
```

Output:

Math	Science	English
85	78	90
72	80	75
88	85	92

b. Using Seaborn:

Seaborn does not directly support tables, but we can display data clearly using heatmaps which act like table charts.

Program:

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

```
import pandas as pd
```

```
# Create DataFrame
```

```
data = {
    'Math': [85, 72, 88],
    'Science': [78, 80, 85],
    'English': [90, 75, 92]
}
```

```
df = pd.DataFrame(data, index=['A', 'B', 'C'])
```

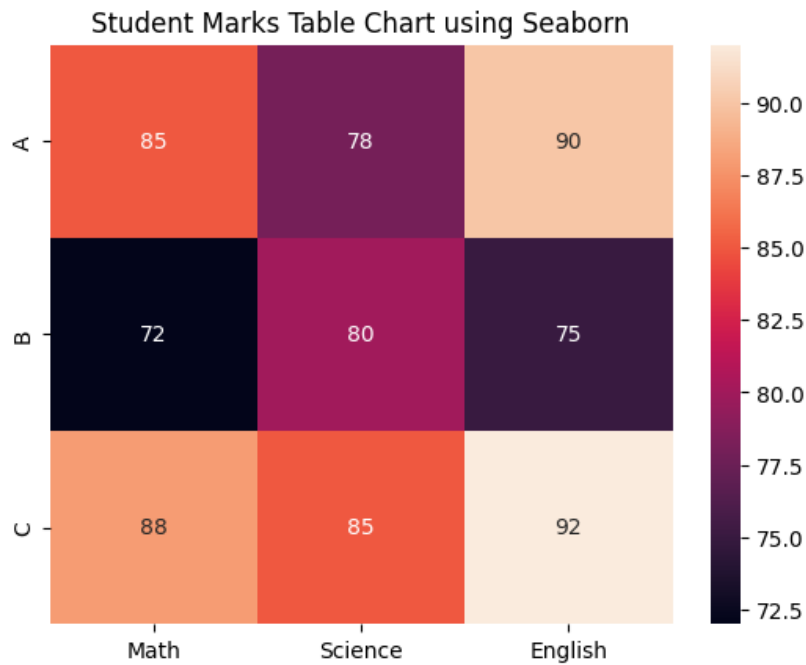
```
# Create heatmap
```

```
sns.heatmap(df, annot=True, fmt="d")
```

```
plt.title("Student Marks Table Chart using Seaborn")
```

```
plt.show()
```

Output:



Topic 8 - Polar Chart

A polar chart is a type of graph where data is shown using angles and distances from the center, instead of the usual x-axis and y-axis.

Instead of straight horizontal and vertical lines:

- Data is plotted in a circular format
- Points move around a circle (angle) and away from the center (radius)

Think of it like a clock:

- The direction (angle) shows where the data lies
- The distance from the center shows how big the value is

A polar chart:

- Uses angles (θ) instead of x-axis
- Uses distance from center (radius r) instead of y-axis
- Looks like a circular graph

SSS

- The center of the chart is called the origin
- The chart uses two values:

- o Angle (θ / theta) direction around the circle
- o Radius (r) distance from the center
- Angles are usually measured in radians
- Polar charts are best when data is:
 - o Cyclic (repeats in a circle)
 - o Direction-based

Polar charts are commonly used in:

- Wind direction and speed analysis
- Radar and signal processing

Program:

a. Using Matplotlib:

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
# Step 1: Create angle values (in radians)
```

```
angles = np.linspace(0, 2 * np.pi, 8)
```

```
# Step 2: Create radius values
```

```
radius = [1, 2, 3, 4, 3, 2, 1, 2]
```

```
# Step 3: Create a polar plot
```

```
plt.figure()
```

```
plt.polar(angles, radius)
```

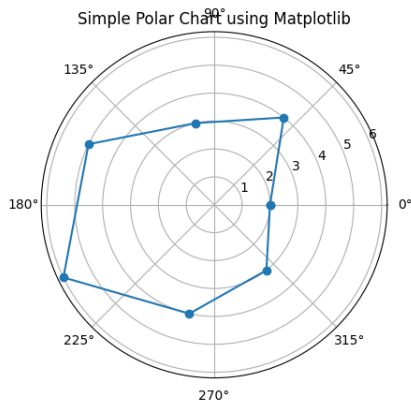
```
# Step 4: Add title
```

```
plt.title("Simple Polar Chart Example")
```

```
# Step 5: Show the chart
```

```
plt.show()
```

Output:



Note: Seaborn does not directly support polar charts

Topic 9 – Histogram

- A histogram is a type of graph used to show how data is distributed.

It tells us how many values fall into different ranges.

- Represents continuous numerical data
- Data is divided into bins (ranges of values)
- X-axis represents data intervals
- Y-axis represents frequency or count
- Bars touch each other, showing continuity
- Height of each bar indicates how often values occur

Parts of a Histogram

- X-axis: Value ranges (bins)
- Y-axis: Frequency (count of values)
- Bars: Show how many values fall in each range

Program

a. Using Matplotlib:

```
import matplotlib.pyplot as plt
```

```
# Sample data
```

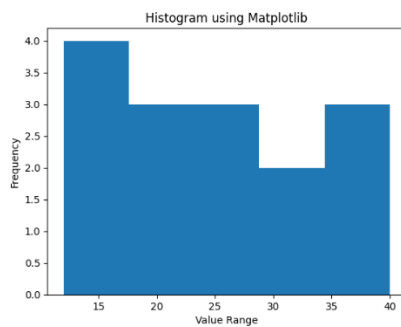
```
data = [12, 15, 14, 16, 18, 20, 22, 25, 25, 28, 30, 32, 35, 35, 40]
```

```
# Create histogram
plt.hist(data, bins=5)

# Add labels and title
plt.xlabel("Value Range")
plt.ylabel("Frequency")
plt.title("Histogram using Matplotlib")

# Show plot
plt.show()
```

Output:



b. Using Seaborn:

```
import seaborn as sns
import matplotlib.pyplot as plt

# Sample data
data = [12, 15, 14, 16, 18, 20, 22, 25, 25, 28, 30, 32, 35, 35, 40]

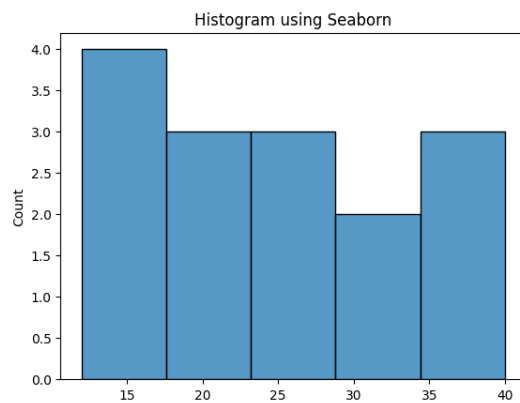
# Create histogram
sns.histplot(data, bins=5)

# Add title
plt.title("Histogram using Seaborn")
```

```
# Show plot
```

```
plt.show()
```

Output:



Topic 10 - Lollipop chart

A lollipop chart is a type of data visualization that is similar to a bar chart, but instead of solid bars, it uses lines with dots at the end.

The line represents the value, and the dot at the end looks like a lollipop, which is how the chart gets its name.

Structure of a Lollipop Chart

A lollipop chart consists of:

- Categories on one axis (e.g., names, types, days)
- Numerical values on the other axis
- Thin lines (sticks) representing magnitude
- Circles (lollipops) at the end showing exact values

Program:

a. Using matplotlib:

```
import matplotlib.pyplot as plt
```

```
# Data
categories = ['A', 'B', 'C', 'D', 'E']
values = [10, 25, 15, 30, 20]

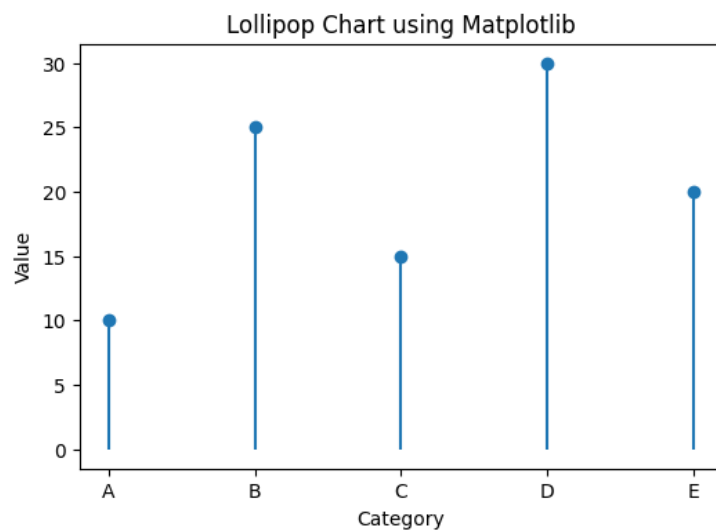
# Create plot
plt.figure(figsize=(6, 4))

# Draw lines
plt.stem(categories, values, basefmt=" ")

plt.title("Lollipop Chart using Matplotlib")
plt.xlabel("Category")
plt.ylabel("Value")

plt.show()
```

Output:



b. Using seaborn

Seaborn cannot directly implement lollipop chart, but it uses a mix of matplotlib library code.

i. Horizontal lolipop

```
import matplotlib.pyplot as plt
import seaborn as sns

# Data
categories = ['A', 'B', 'C', 'D', 'E']
values = [10, 25, 15, 30, 20]

plt.figure(figsize=(6, 4))

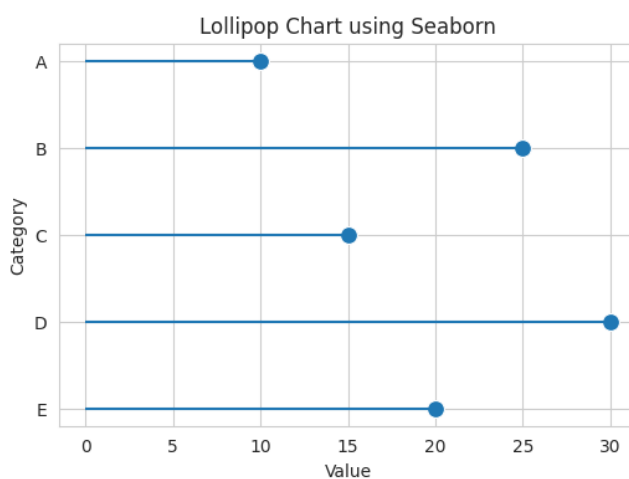
# Draw horizontal lines (sticks)
plt.hlines(y=categories, xmin=0, xmax=values)

# Draw dots (heads)
sns.scatterplot(x=values, y=categories, s=100)

plt.title("Lollipop Chart using Seaborn")
plt.xlabel("Value")
plt.ylabel("Category")

plt.show()
```

Output:



ii. Vertical lolipop

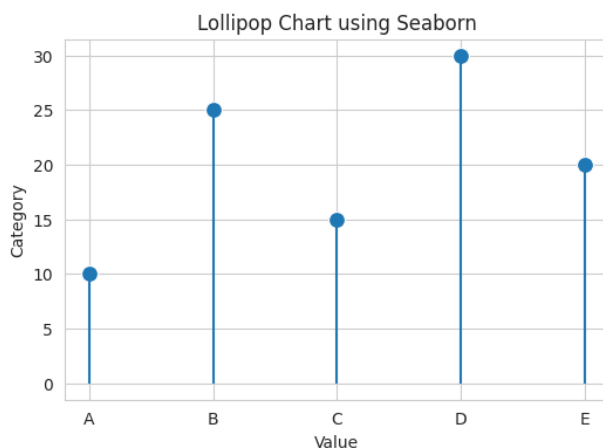
```
import matplotlib.pyplot as plt
import seaborn as sns

# Data
categories = ['A', 'B', 'C', 'D', 'E']
values = [10, 25, 15, 30, 20]

plt.figure(figsize=(6, 4))
# Draw horizontal lines (sticks)
plt.vlines(x=categories, ymin=0, ymax=values)
# Draw dots (heads)
sns.scatterplot(x=categories, y=values, s=100)
plt.title("Lollipop Chart using Seaborn")
plt.xlabel("Value")
plt.ylabel("Category")

plt.show()
```

Output:



Topic 11 - Choosing best chart

Choosing the best chart is an important step in data visualization. There is no fixed rule that tells us which chart should be used for every dataset. However, some

general guidelines help in selecting an appropriate chart so that the data is presented in a clear, meaningful, and understandable way. The main goal of visualization is to convey the correct information without confusing or misleading the audience .

Key Guidelines for Choosing the Best Chart

- The type of data must be understood first.
 - o For continuous data, charts like histograms are suitable.
 - o For categorical or ranking data, bar charts or lollipop charts are more appropriate .
- The chosen chart should clearly represent the information without distorting facts.

A good chart should highlight only the relevant data needed by the audience .

- Simplicity is preferred over complexity.

Simple charts are easier to understand than complicated charts that require long explanations or additional text .

- The chart should not overload the audience with too much information.

The purpose of visualization is to make data easy to interpret, not confusing .

Chart Selection Based on Purpose

According to the textbook, charts can be selected based on what we want to show:

1. To Show Distribution

- Histogram
- Density plot
- Box plot
- Violin plot

These charts are useful to understand how data values are spread .

2. To Show Correlation or Relationship

- Scatter plot
- Scatter plot with line of best fit
- Bubble plot

These charts help in analyzing the relationship between two or more variables .

3. To Show Change Over Time

- Line chart
- Time series plot
- Stacked area chart

These charts are useful when data changes with respect to time .

4. To Show Composition

- Pie chart
- Bar chart
- Treemap

These charts show how different parts contribute to a whole .

5. To Show Ranking or Comparison

- Ordered bar chart
- Lollipop chart
- Dot plot

These charts are useful when data needs to be compared or ranked .

Topic 12 - Other Libraries to explore.

Apart from Matplotlib and Seaborn, there are several other Python libraries available for data visualization. These libraries help in creating interactive, advanced, and visually appealing graphs. They are useful in exploratory data analysis (EDA), research, and dashboard development.

Plotly

- Plotly is a web-based visualization library
- It is used to create interactive charts
- Supports 2D and 3D visualizations
- Can be easily used in Jupyter Notebook
- Suitable for real-time and dashboard-based applications

Example: Interactive line charts, scatter plots, 3D graphs

ggplot

- ggplot is based on the Grammar of Graphics concept
- It is inspired by the R programming language
- Helps in creating statistical and analytical plots
- Uses a layer-by-layer approach to build graphs
- Commonly used in data analysis and academic research

Altair

- Altair is a declarative statistical visualization library
- Built on top of Vega-Lite
- Uses simple and readable syntax
- Helps in quick exploratory data analysis
- Produces interactive and clean graphs